Práctica: Gestión de Personas de un Centro Educativo

📚 Documentación oficial recomendada

Todo el ejercicio se basa exclusivamente en APIs estándar de **Java SE (Oracle)**.
Consulta la documentación oficial para resolver dudas conceptuales y técnicas:

- Tutoriales Java (Oracle): https://docs.oracle.com/javase/tutorial/
- API Java SE 8 (muy usada en docencia): https://docs.oracle.com/javase/8/docs/api/

---

1. Objetivo

Implementar una aplicación **por consola** que gestione un conjunto de personas de un centro educativo mediante:

- **POO**: diseño de clases y herencia\
- **Colecciones**: `ArrayList` (Nivel 1) y `Set` (Nivel 2)\
- **Validación con RegEx**\
- **Fechas**: `java.time`\
- **Recursividad**

---

2. Reglas generales

Si un dato es inválido → lanzar `IllegalArgumentException`.

🔗 Documentación: - Excepciones: https://docs.oracle.com/javase/tutorial/essential/exceptions/ - IllegalArgumentException: https://docs.oracle.com/javase/8/docs/api/java/lang/IllegalArgumentException.html

---

3. Modelo de datos y POO

Persona (clase abstracta)

Conceptos:

- Clases
- Herencia
- Encapsulación
- Clases abstractas
- Constructores

🔗 Documentación oficial:

- Clases: https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html
- Constructores: https://docs.oracle.com/javase/tutorial/java/javaOO/constructors.html
- Herencia: https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html

- Clases abstractas: https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html
- Modificadores de acceso: https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

---

4. Validaciones (RegEx)

Se validan email y documento mediante expresiones regulares.

🔗 Documentación oficial:

- Tutorial de expresiones regulares: https://docs.oracle.com/javase/tutorial/essential/regex/
- Paquete java.util.regex: https://docs.oracle.com/javase/8/docs/api/java/util/regex/package-summary.html
- Clase Pattern: https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html

---

5. Colecciones

Se utilizan `estructuras de datos` de la `Java`.

- ArrayList
- Set (HashSet) para evitar duplicados

🔗 Documentación oficial:

- Introducción a colecciones: https://docs.oracle.com/javase/tutorial/collections/intro/
- Interface Collection: https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html
- List: https://docs.oracle.com/javase/8/docs/api/java/util/List.html
- ArrayList: https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html
- Set: https://docs.oracle.com/javase/8/docs/api/java/util/Set.html
- HashSet: https://docs.oracle.com/javase/8/docs/api/java/util/HashSet.html

---

6. Fechas (API java.time)

Se utilizará la API moderna introducida en Java 8.

Conceptos:

- LocalDate
- Period
- Fecha actual

- Cálculo de edad

🔗 Documentación oficial:

- Tutorial Date-Time API: https://docs.oracle.com/javase/tutorial/datetime/
- LocalDate: https://docs.oracle.com/javase/8/docs/api/java/time/LocalDate.html
- Period: https://docs.oracle.com/javase/8/docs/api/java/time/Period.html

---

7. Recursividad

Búsqueda sin usar bucles.

🔗 Documentación relacionada:

- Métodos en Java: https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html

(Recurso teórico adicional) - Recursión: https://en.wikipedia.org/wiki/Recursion_(computer_science)

---

8. Gestión de datos --- CentroEducativo

Se gestionará un conjunto de personas en memoria.

Operaciones:

- Registrar persona
- Listar personas
- Buscar por id
- Buscar por documento
- Búsqueda recursiva por prefijo

🔗 Documentación útil:

- Interface List: https://docs.oracle.com/javase/8/docs/api/java/util/List.html
- Streams (ampliación opcional): https://docs.oracle.com/javase/8/docs/api/java/util/stream/package-summary.html

---

9. Objetivo e implementación

9.1 Objetivo

El objetivo es implementar una aplicación **por consola** que gestione un conjunto de personas de un centro educativo mediante:

- **POO**: diseño de clases y **herencia** (`Persona`, `Alumno`, `Profesor`).

- **Colecciones**: `ArrayList` (Nivel 1) y `Set` (Nivel 2).
- **Validación con RegEx**: documento e email.
- **Fechas**: `java.time` (`LocalDate`, `Period`).
- **Recursividad** (Nivel 2): búsqueda sin bucles.

9.2. Reglas generales (obligatorias)

- Todas las clases deben tener **constructores**, **getters**, y `toString()` adecuados.
- Si un dato es inválido, se debe lanzar `IllegalArgumentException` con un mensaje claro.
- Las comparaciones de `documento` y `email` se harán **ignorando espacios** y con normalización:
  - `documento = documento.trim().toUpperCase()`
  - `email = email.trim()` (opcionalmente `toLowerCase()` si así lo decides, pero entonces **siempre** igual).

9.3. Modelo de datos y contratos (interfaces "de hecho")

9.3.1. `Persona` (clase abstracta)

Paquete: `com.docencia.model`
Responsabilidad: representar una persona genérica con datos comunes.

Atributos (mínimos)

- `private final int id;`
- `private String nombre;`
- `private String documento;`
- `private String email;`
- `private LocalDate fechaNacimiento;`
- `private final LocalDate fechaRegistro;`

> Nota: `id` y `fechaRegistro` se consideran inmutables una vez creada la persona.

Constructores (contrato)

```
public Persona(int id, String nombre, String documento, String email,
               LocalDate fechaNacimiento, LocalDate fechaRegistro)
```

Precondiciones (VALIDAR):

- `id > 0`
- `nombre != null` y `nombre.trim().length() >= 2`
- `documento` válido por RegEx (ver `Validaciones.documentoValido`)
- `email` válido por RegEx (ver `Validaciones.emailValido`)
- `fechaNacimiento != null` y `fechaNacimiento.isBefore(LocalDate.now())`
- `fechaRegistro != null` y `!fechaRegistro.isAfter(LocalDate.now())`
  (puede ser hoy o anterior)

```
public Persona(int id)
```

Precondiciones (VALIDAR):

- `id > 0`

Postcondiciones:

- La instancia queda creada con los datos almacenados y normalizados.
- `fechaRegistro` queda establecida y no cambia, es `inmutable`.

Excepciones:

- `IllegalArgumentException` si alguna precondición falla.

> Alternativa permitida (más simple): no recibir `fechaRegistro` y fijarla como `LocalDate.now()` dentro del constructor.
> Si eliges esa alternativa, documenta y crea el constructor equivalente.

Métodos obligatorios (contratos)

```
public final int getId()
```

- **Devuelve** el id (>0).

```
public String getNombre()
public void setNombre(String nombre)
```

- `setNombre`: valida que cumpla las mismas reglas del constructor.

```
public String getDocumento()
public void setDocumento(String documento)
```

- `setDocumento`: normaliza y valida por RegEx.

```
public String getEmail()
public void setEmail(String email)
```

- `setEmail`: valida por RegEx.

```
public LocalDate getFechaNacimiento()
public void setFechaNacimiento(LocalDate fechaNacimiento)
```

- `setFechaNacimiento`: debe ser anterior a hoy.

```
public final LocalDate getFechaRegistro()
```

- Devuelve la fecha de registro (no nula).

```
public int getEdad()
```

Contrato:

- **Devuelve** años cumplidos (>=0) calculados a partir de la fecha de nacimiento.

```
public abstract String getTipo()
```

- Las subclases devuelven `"ALUMNO"` o `"PROFESOR"`.

```
@Override public String toString()
```

- Debe incluir: tipo, id, nombre, documento, email, fechaNacimiento, fechaRegistro, y campos extra de la subclase.

9.3.2. `Alumno` (subclase de `Persona`)

Paquete: `com.docencia.model`

Atributos extra

- `private String curso;`
- **Nivel 2:** `private Set<String> modulos;` (sin duplicados)

Constructores (contrato)

```
public Alumno(int id, String nombre, String documento, String email,
              LocalDate fechaNacimiento, LocalDate fechaRegistro,
              String curso)
public Alumno(int id)
```

Precondiciones:

- Todas las de `Persona`
- `curso != null` y `el tamaño del curso debe de ser mayor a 3 caracteres`

Postcondiciones:

- `getTipo()` devuelve `"ALUMNO"`

Métodos obligatorios

```
@Override public String getTipo() // "ALUMNO"
```

```
public String getCurso()
public void setCurso(String curso)
```

Funciones que se deben de implementar:

```
public boolean addModulo(String modulo)
public boolean removeModulo(String modulo)
public Set<String> getModulos()
```

Contrato:

- `addModulo`:
  - pre: `modulo != null` y `el tamaño del curso debe de ser mayor a 2 caracteres` en caso contrario no lo añade.
  - post: devuelve `true` si se añadió (no existía), `false` si ya estaba.
- `getModulos`:
  - devuelve una el conjunto de modulos sepados por comas`

9.3.3. `Profesor` (subclase de `Persona`)

Paquete sugerido: `com.docencia.model`

Atributos extra

- `private String departamento;`

Constructores (contrato)

```
public Profesor(int id, String nombre, String documento, String email,
                LocalDate fechaNacimiento, LocalDate fechaRegistro,
                String departamento)
public Profesor(int id)
```

Precondiciones:

- Todas las de `Persona`
- `departamento != null` y `el tamaño del curso debe de ser mayor a 4 caracteres`

Métodos obligatorios

```
@Override public String getTipo() // "PROFESOR"
public String getDepartamento()
public void setDepartamento(String departamento)
```

9.4. Validaciones (RegEx) y contratos

Clase `Validaciones` (utilidades estáticas)

Paquete sugerido: `com.docencia.util`

Expresiones regulares (mínimas)

- **Email:**
  - `valida el correcto formato del campo de tipo texto. ej: ana@ejemplo.xom`
- **Documento (genérico):**
  - `valida el correcto formato de un documento tipo dni`

Métodos (contratos)

```
public static boolean emailValido(String email)
```

- pre: `email != null`
- post: devuelve `true` si cumple RegEx, `false` en caso contrario

```
public static boolean documentoValido(String documento)
```

- pre: `documento != null`
- post: devuelve `true` si cumple RegEx, `false` en caso contrario

> Recomendación: normalizar antes de validar el eliminando espacios y mayúsculas/minúsculas.

9.5. Gestión de colecciones: `CentroEducativo` y contratos

Paquete: `com.docencia.service`

9.5.1. Atributos

```
private List<Persona> personas;
```

Invariante:

- No debe haber dos personas con el mismo `id` / `documento` / `email`.

9.5.2. Atributos

```
private Set<String> documentosRegistrados;
private Set<String> emailsRegistrados;
```

Importante:

- No hay duplicados de `documento` ni de `email`.

9.5.3. Constructor (contrato)

```
public CentroEducativo()
```

- inicializa `personas = new ArrayList<>()` o utiliza `Set` con `HashSet`, según consideres más adecuado, teniendo encuenta que no queres elementos duplicados.

9.5.4. Métodos mínimos (contratos)

```
public void registrarPersona(Persona persona)
```

Precondiciones:

- `persona != null`
- `id` no repetido (buscar en `personas`)
- `documento` y `email` no repetidos (consultar Set)

Postcondiciones:

- Se añade `persona` a la lista
- Se añade su `documento` a `documentosRegistrados` y su `email` a `emailsRegistrados`.

Excepciones:

- `IllegalArgumentException` si `persona` es nulo o si hay duplicados con un `mensaje adecuado` en el error.

```
public List<Persona> listarPersonas()
```

- Devuelve una lista con todas las personas (puede ser copia: `List<>(personas)`)

```
public Persona buscarPorId(int id)
```

- pre: `id > 0`
- post: devuelve la persona con ese id o `null` si no existe.

```
public Persona buscarPorDocumento(String documento)
```

- pre: `documento != null`
- post: devuelve la persona con ese documento o `null`.

```
public List<Alumno> listarAlumnos()
public List<Profesor> listarProfesores()
```

- post: devuelven listas filtradas por tipo.

9.5.5. Recursividad (vamos a aplicarla)

```
public List<Persona> buscarPorPrefijo(String prefijo)
```

Contrato:

- pre: `prefijo != null/no vacio/ tamaño adecuado`
- post: devuelve todas las personas cuyo nombre empieza por `prefijo` (ignorando mayúsculas/minúsculas).
- restricción: el recorrido debe hacerse **sin bucles**. Puedes utilizar un método auxiliar

Auxiliar sugerido pero no la única solución:

```java
private void buscarRec(int index, String prefijo, ArrayList<Persona>
resultado)
```

- caso base: `index >= personas.size()` → termina
- caso recursivo: evalúa `personas.get(index)` y llama con `index + 1`

9.6. Consola (Main) y contratos mínimos

Paquete: `com.docencia.app`

Menú mínimo

1. Registrar alumno
2. Registrar profesor
3. Listar personas
4. Buscar por id
5. Buscar por prefijo (recursivo)
6. Listar mayores de X
7. Salir

Contrato de entrada/salida:

- Si el usuario introduce un dato inválido, se debe mostrar un mensaje y volver a pedirlo .

9.7. Criterios de evaluación

- Herencia correcta (`Persona` abstracta + `Alumno` + `Profesor`)
- `ArrayList` funcional con altas y consultas
- Validación con RegEx (email/documento)
- Uso correcto de `LocalDate` y cálculo de edad
- `Set` para evitar duplicados de documento y email
- `Set` de módulos en `Alumno` (sin repetidos)
- Búsqueda recursiva por prefijo sin bucles
- Código modular (paquetes, clases utilitarias, mensajes claros)
- Menú funcional para probar la aplicación

9.8. Estructura recomendada del proyecto

```
src/
  com.docencia.app/
    Main.java
  com.docencia.model/
    Persona.java
    Alumno.java
    Profesor.java
```

```
    com.docencia.service/
      CentroEducativo.java
    com.docencia.util/
      Validaciones.java
```